



昆明理工大学

Kunming University of Science and Technology

总体刚度矩阵程序设计作业报告

姓 名: 陆晓宝

学 号: 20252110009

专 业: 工程力学

班 级: 力学 3 班

指导老师: 肖姚

1. 引言

本次作业对应《系统总体刚度方程》章节内容，旨在通过独立编写程序，掌握杆单元和二维桁架单元有限元分析的完整流程。核心任务包括：根据单元连接关系生成对号矩阵 LM 、实现总体刚度矩阵的直接组装、施加位移边界条件并求解总体刚度方程、计算单元应力和轴力。

有限元法是工程力学、岩土工程、结构工程以及航空航天等领域广泛采用的重要数值分析方法。其基本思想是将复杂连续体划分为若干有限单元，在单元层面建立力学关系，然后通过总体组装形成整个结构的控制方程。

在有限元分析过程中，总体刚度矩阵的建立是最关键的步骤之一。总体刚度矩阵不仅反映结构整体刚度特性，而且直接影响求解精度和计算效率。对于实际工程结构而言，总体刚度矩阵通常具有对称性、稀疏性以及半正定性等特点。

通过本次作业，深入理解有限元方法“离散化 - 单元分析 - 整体组装 - 方程求解 - 后处理”的核心思想，掌握总体刚度矩阵的物理意义与基本性质，为后续复杂结构的有限元分析奠定基础。

2. 总体刚度方程的建立过程

总体刚度方程描述了结构整体的节点位移与节点载荷之间的关系，其建立基于节点平衡条件和位移协调条件两个基本原理，具体步骤如下：

2.1 单元刚度方程

对于任意一个杆单元或桁架单元，在局部坐标系下的单元刚度方程为：

$$k^e d^e = f^e$$

结构离散：将连续结构划分为若干个杆/桁架单元，单元之间通过节点连接，定义节点坐标、单元连接关系、材料参数与截面参数。

单元分析：在局部坐标系下推导单元刚度矩阵 k^e ，描述单元节点力与节点位移的关系 $\{f\}^e = [k]^e \{d\}^e$ 。对于二维桁架单元，需通过坐标转换矩阵 $[T]$ 将局部坐标系下的单元刚度矩阵转换为全局坐标系下的单元刚度矩阵 $[K]^e = [T]^T [k]^e [T]$ 。

总体组装：根据位移协调条件，同一节点在不同单元中的位移相同，利用对号矩阵 **LM** 将各单元刚度矩阵的元素“对号入座”累加至总体刚度矩阵 $[K]$ 中，得到未施加边界条件的总体刚度方程：

$$[K]\{d\} = \{F\}$$

边界条件施加：引入位移边界条件，消除总体刚度矩阵的奇异性，求解未知节点位移。

后处理：根据节点位移计算单元应力、轴力及约束反力。

2.2 单元刚度矩阵推导

2.2.1 一维杆单元

局部坐标系下，一维杆单元的刚度矩阵为：

$$[k]^e = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

其中，E 为弹性模量，A 为截面积，L 为单元长度。

2.2.2 二维桁架单元

局部坐标系下的单元刚度矩阵与一维杆单元相同，全局坐标系下的单元刚度矩阵为：

$$[K]^e = \frac{EA}{L} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}$$

其中， $c = \cos\theta = \frac{x_2 - x_1}{L}$ ， $s = \sin\theta = \frac{y_2 - y_1}{L}$ 为单元轴线相对于全局

坐标系的方向余弦， θ 为单元轴线与 x 轴的夹角。

3、程序实现

程序采用 Python 语言编写，基于 NumPy 库进行矩阵运算，严格按照模块化设计原则划分为 5 个核心函数，对应有限元求解的 5 个步骤。程序内部自由度编号从 0 开始，自动处理输入文件中从 1 开始的编号转换。

3.1 前处理模块

前处理模块负责读取 JSON 格式的输入文件，解析并存储以下模型数据：

模型基本信息：维度 nsd 、节点自由度 $ndof$ 、节点数 nnp 、单元数 nel 、单元节点数 nen

材料与截面参数：弹性模量数组 E 、截面积数组 $CArea$

几何信息：节点坐标数组 x 、 y ，单元连接数组 IEN

边界条件：固定自由度数组 $fixed_dof$ 、固定位移值数组 $fixed_value$

载荷信息：载荷自由度数组 $force_dof$ 、载荷值数组 $force_value$

3.2 对号矩阵 LM 的生成方法

对号矩阵 LM 用于建立单元局部自由度与全局自由度的映射关系，其生成逻辑如下：

对于任意单元 e ，其连接的两个全局节点编号为 $i = IEN[e][0]$ 、 $j = IEN[e][1]$ （输入为 1 开始，转换为 0 开始后为 $i-1$ 、 $j-1$ ）。

每个节点对应 $ndof$ 个全局自由度，节点 p （0 开始）的全局自由度编号为： $p \times ndof, p \times ndof + 1, \dots, p \times ndof + ndof - 1$ 。

单元 e 的 LM 矩阵为两个节点的全局自由度编号的拼接，即：

$LM[e] = [i \times ndof, i \times ndof + 1, j \times ndof, j \times ndof + 1]$ （二维桁架单元， $ndof = 2$ ）。

示例：算例 2 中单元 1 连接节点 1 和 3（输入编号），转换为 0 开始为 0 和 2，其 LM 矩阵为 $[0,1,4,5]$ ；单元 2 连接节点 2 和 3，转换为 0 开始为 1 和 2，其 LM 矩阵为 $[2,3,4,5]$ 。

3.3 直接组装算法

直接组装算法基于叠加原理，将每个单元刚度矩阵的元素按照 LM 矩阵的映射关系累加至总体刚度矩阵的对应位置，核心代码逻辑如下：

```
# 初始化总体刚度矩阵为零矩阵
K = np.zeros((nnp*ndof, nnp*ndof))
for e in range(nel):
    # 计算单元 e 的全局刚度矩阵 Ke
    Ke = compute_element_stiffness(e, x, y, E, CArea, ndof)
    # 获取单元 e 的对号矩阵
    lm = LM[e]
    # 对号入座累加
    for a in range(nen*ndof):
        for b in range(nen*ndof):
            K[lm[a], lm[b]] += Ke[a, b]
```

该算法的核心优势是逻辑清晰、易于实现，且能够自动处理任意数量的单元与节点。

3.4 位移边界条件处理方法

程序核心实现了缩减法处理位移边界条件，同时实现了修改法作为对比。

3.4.1 缩减法

将总体位移向量 $\{d\}$ 划分为已知位移 $\{d_E\}$ 和未知位移 $\{d_F\}$ 两部分，对应的总体刚度矩阵与载荷向量也进行分块：

$$\begin{bmatrix} K_{EE} & K_{EF} \\ K_{FE} & K_{FF} \end{bmatrix} \begin{Bmatrix} d_E \\ d_F \end{Bmatrix} = \begin{Bmatrix} F_E + R_E \\ F_F \end{Bmatrix}$$

其中， R_E 为约束反力， F_E 为已知位移自由度上的外载荷。

求解未知位移：

$$K_{FF}\{d_F\} = \{F_F\} - K_{FE}\{d_E\}$$

重构完整的总体位移向量 $\{d\}$ 。

计算约束反力：

$$\{R_E\} = K_{EE}\{d_E\} + K_{EF}\{d_F\} - \{F_E\}$$

3.4.2 方法对比

缩减法：精度最高，无近似误差，但需要对矩阵进行分块与重新编号，代码实现稍复杂。

修改法：将总体刚度矩阵中已知位移自由度对应的行和列置零，对角元置 1，载荷向量对应位置置为已知位移值，实现简单，但会破坏总体刚度矩阵的对称性与稀疏性。

罚函数法：在已知位移自由度对应的对角元上乘以一个很大的罚数（如 10^{10} ），载荷向量对应位置置为罚数乘以已知位移值，实现最简单，但罚数过大可能导致数值病态。

3.5 后处理模块

后处理模块根据求解得到的总体位移向量，提取每个单元的节点位移 $\{d\}^e$ ，计算单元的长度、方向余弦、应力与轴力：

$$\text{一维杆单元应力: } \sigma = \frac{E}{L}[-1, 1]\{d\}^e$$

$$\text{二维桁架单元应力: } \sigma = \frac{E}{L}[-c, -s, c, s]\{d\}^e$$

$$\text{单元轴力: } N = \sigma A$$

四、验证算例与结果校核

4.1 算例 1：一维两单元杆结构

4.1.1 输入数据

节点坐标：[0,1,2]（单位：m）

单元连接：单元 1（1-2）、单元 2（2-3）

材料参数： $E_1A_1/L_1=100$ 、 $E_2A_2/L_2=200$ （单位：N/m）

边界条件：节点 1 固定（ $d_1=0$ ）

节点载荷：节点 3 受轴向力 $F_3=10$ （单位：N）

4.1.2 程序输出结果

总体刚度矩阵（施加边界条件前）：

$$K = \begin{bmatrix} 100 & -100 & 0 \\ -100 & 300 & -200 \\ 0 & -200 & 200 \end{bmatrix}$$

与理论值完全一致，且满足对称性（ $K=K^T$ ）。

奇异性检查：

施加边界条件前： $\det(K)=0$ ，矩阵奇异，符合无约束结构存在刚体位移的特性。

施加边界条件后：缩减矩阵为 $\begin{bmatrix} 300 & -200 \\ -200 & 200 \end{bmatrix}$ ， $\det=20000 \neq 0$ ，矩阵非奇异。

节点位移： $d_1=0, d_2=0.1, d_3=0.15$ 与理论值完全一致。

约束反力：节点 1 的约束反力 $R_1=-10$ （负号表示与载荷方向相反），大小为 10，与外载平衡。

单元应力与轴力：

单元编号	长度(m)	应力(Pa)	轴力(N)
1	1	10	10
2	1	10	10

结果符合静力学平衡条件。

4.2 算例 2：二维两杆桁架结构

4.2.1 输入数据

```
{
  "Title": "2D truss example",
  "nsd": 2,
  "ndof": 2,
  "nnp": 3,
  "nel": 2,
  "nen": 2,
  "E": [1.0, 1.0],
  "CArea": [1.0, 1.0],
  "x": [1.0, 0.0, 1.0],
  "y": [0.0, 0.0, 1.0],
  "IEN": [[1, 3], [2, 3]],
  "fixed_dof": [1, 2, 3, 4],
  "fixed_value": [0.0, 0.0, 0.0, 0.0],
  "force_dof": [5, 6],
  "force_value": [10.0, 0.0]
}
```

4.2.2 程序输出结果

对号矩阵 LM:

$$LM = \begin{bmatrix} 0 & 1 & 4 & 5 \\ 2 & 3 & 4 & 5 \end{bmatrix}$$

生成正确。

总体刚度矩阵（施加边界条件前）：

$$K = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 \\ 0 & 0 & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} \\ 0 & 0 & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} \\ 0 & 0 & -\frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \\ 0 & -1 & -\frac{\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} & 1 + \frac{\sqrt{2}}{4} \end{bmatrix}$$

满足对称性（ $K = K^T$ ），且 $\det(K) = 0$ ，矩阵奇异。

节点位移： $u_3 = 38.284271$ ， $v_3 = -10.000000$ 。与理论值误差小于 10^{-6} ，符合要求。

单元计算结果：

单元编号	长度(m)	方向余弦	应力(Pa)	轴力(N)
1	1.0	(0, 1)	-10.0000	-10.00
2	1.4142	(0.7071, 0.7071)	14.1421	14.1421

与理论值完全一致。

约束反力：

节点 1: $R_{x1} = -10$, $R_{y1} = 10$; 节点 2: $R_{x2} = 0$, $R_{y2} = -10$ 。所有约束反力的合力与外载荷平衡。

五、总体刚度矩阵性质分析

5.1 对称性

总体刚度矩阵是对称矩阵，即 $K_{ij} = K_{ji}$ 。

理论依据：单元刚度矩阵本身是对称矩阵，直接组装过程是对称累加，因此总体刚度矩阵保持对称性。

物理意义：符合功的互等定理，即节点 i 施加单位力引起节点 j 的位移，等于节点 j 施加单位力引起节点 i 的位移。

程序验证：两个算例的总体刚度矩阵均满足 $K = K^T$ ，最大误差小于 10^{-15} （数值计算误差）。

5.2 奇异性

未施加足够位移边界条件时，总体刚度矩阵是奇异矩阵（ $\det(K) = 0$ ），施加边界条件后缩减矩阵非奇异。

理论依据：无约束结构可以发生刚体位移，此时应变能为零，即 $0.5 d^T K d = 0$ ，存在非零位移解，因此矩阵奇异。施加足够的约束，消除所有刚体位移后，位移解唯一，矩阵非奇异。

程序验证：两个算例施加边界条件前 $\det(K) = 0$ ，施加后缩减矩阵行列式不为零。

5.3 稀疏性

总体刚度矩阵是稀疏矩阵，大部分元素为零。

理论依据：每个单元仅与自身连接的节点相关，因此单元刚度矩阵的元素仅会影响总体刚度矩阵中对应节点自由度的位置。对于二维桁架单元，每个单元仅影响 4 个自由度，因此总体刚度矩阵每行最多有 4 个非零元素。

特点：对于大型结构，稀疏性尤为显著，可通过稀疏矩阵存储与求解技术大幅提高计算效率。

5.4 对角元非负性

总体刚度矩阵的所有对角元均非负，即 $K_{ii} \geq 0$ 。

理论依据：总体刚度矩阵是半正定矩阵，应变能 $U = 0.5 d^T K d \geq 0$ 对任意位移向量 d 成立。当 d 为第 i 个自由度单位位移、其他自由度为零的向量时， $U = 0.5 K_{ii} \geq 0$ ，因此 $K_{ii} \geq 0$ 。

物理意义： K_{ii} 表示在第 i 个自由度产生单位位移、其他自由度约束时，需要施加的力。根据能量守恒，外力做功非负，因此 K_{ii} 非负。

5.5 总体刚度矩阵第 j 列的物理意义

总体刚度矩阵的第 j 列 $\{K\}_j$ 表示：当第 j 个自由度产生单位位移，其他所有自由度均被约束（位移为 0）时，在各个自由度上需要施加的节点力向量。

其中，第 i 个元素 K_{ij} 表示：第 j 个自由度产生单位位移时，在第 i 个自由度上需要施加的力。

示例：算例 1 中总体刚度矩阵的第 3 列 $[0, -200, 200]^T$ ，表示当节点 3 产生单位位移、节点 1 和 2 约束时，需要在节点 2 施加 -200N 的力，在节点 3 施加 200N 的力。

六、结论与心得

本次作业成功完成了杆/桁架结构有限元程序的开发，通过两个标准算例的验证，程序能够准确求解节点位移、约束反力及单元应力。通过本次作业，系统掌握了有限元程序的五大核心步骤，深入理解了总体刚度矩阵的组装原理与物理意义，特别是对号矩阵的作用与边界条件处理的重要性。

在编程过程中，深刻体会到了有限元理论与数值计算的结合，例如总体刚度矩阵的奇异性在数值上表现为行列式为零，而其物理本质是刚体位移的存在。同时，模块化的程序设计方法不仅提高了代码的可读性与可维护性，也为后续扩展三维桁架单元奠定了基础。

七、附录：源代码

```
import numpy as np
import json
import os

# 1. 前处理模块
def load_model(json_path):
    """读取 JSON 模型文件，处理自由度编号转换（输入 1 起始 → 内部 0 起始）"""
    with open(json_path, 'r', encoding='utf-8') as f:
        model = json.load(f)

    # 基础参数
    nsd = model['nsd'] # 空间维度 1/2
    ndof = model['ndof'] # 单节点自由度数
    nnp = model['nnp'] # 节点总数
    nel = model['nel'] # 单元总数
    nen = model['nen'] # 单元节点数

    # 节点坐标
    x = np.array(model['x'], dtype=np.float64)
    y = np.array(model['y'], dtype=np.float64) if nsd >= 2 else None

    # 材料与截面
    E = np.array(model['E'], dtype=np.float64)
    CArea = np.array(model['CArea'], dtype=np.float64)

    # 单元连接数组（输入 1 起始 → 转为 0 起始）
    IEN = np.array(model['IEN'], dtype=int) - 1

    # 边界条件（输入 1 起始 → 转为 0 起始）
    fixed_dof = np.array(model['fixed_dof'], dtype=int) - 1
    fixed_value = np.array(model['fixed_value'], dtype=np.float64)

    # 节点载荷（输入 1 起始 → 转为 0 起始）
    force_dof = np.array(model['force_dof'], dtype=int) - 1
    force_value = np.array(model['force_value'], dtype=np.float64)

    # 总自由度数
    n_total_dof = nnp * ndof

    # 组装全局载荷向量
```

```

F = np.zeros(n_total_dof, dtype=np.float64)
for i, dof in enumerate(force_dof):
    F[dof] = force_value[i]

return {
    'title': model['Title'],
    'nsd': nsd, 'ndof': ndof, 'nnp': nnp, 'nel': nel, 'nen': nen,
    'x': x, 'y': y, 'E': E, 'CArea': CArea,
    'IEN': IEN, 'fixed_dof': fixed_dof, 'fixed_value':
fixed_value,
    'F': F, 'n_total_dof': n_total_dof
}

```

2. 对号矩阵 LM 生成

```

def build_LM(IEN, ndof, nen):
    """根据单元连接数组 IEN (0 起始) 生成对号矩阵 LM
    LM[e, :] 存储第 e 个单元所有局部自由度对应的全局自由度编号
    """
    nel = IEN.shape[0]
    LM = np.zeros((nel, nen * ndof), dtype=int)
    for e in range(nel):
        for i in range(nen):
            node_id = IEN[e, i]
            # 节点 node_id 对应的全局自由度起始编号
            start = node_id * ndof
            LM[e, i * ndof: (i + 1) * ndof] = np.arange(start, start +
ndof)
    return LM

```

3. 单元刚度矩阵计算

```

def compute_element_stiffness(e, model):
    """计算第 e 个单元的全局坐标系下的刚度矩阵"""
    x = model['x']
    y = model['y']
    E = model['E'][e]
    A = model['CArea'][e]
    IEN = model['IEN']
    ndof = model['ndof']
    nsd = model['nsd']

    node_i = IEN[e, 0]
    node_j = IEN[e, 1]

```

```

# 计算单元长度与方向余弦
dx = x[node_j] - x[node_i]
if nsd == 1:
    L = abs(dx)
    c = dx / L
    s = 0
else:
    dy = y[node_j] - y[node_i]
    L = np.sqrt(dx ** 2 + dy ** 2)
    c = dx / L
    s = dy / L

# 单元轴向刚度
k0 = E * A / L

# 全局坐标系下单元刚度矩阵
if nsd == 1:
    Ke = k0 * np.array([
        [1, -1],
        [-1, 1]
    ])
else:
    Ke = k0 * np.array([
        [c ** 2, c * s, -c ** 2, -c * s],
        [c * s, s ** 2, -c * s, -s ** 2],
        [-c ** 2, -c * s, c ** 2, c * s],
        [-c * s, -s ** 2, c * s, s ** 2]
    ])

return Ke, L, c, s

# 4. 总体刚度矩阵直接组装
def assemble_global_stiffness(LM, Ke_list, n_total_dof):
    """根据对号矩阵 LM, 将单元刚度矩阵累加组装为总体刚度矩阵"""
    K = np.zeros((n_total_dof, n_total_dof), dtype=np.float64)
    nel = LM.shape[0]

    for e in range(nel):
        Ke = Ke_list[e]
        lm = LM[e]
        # 对号入座累加
        for a in range(len(lm)):

```



```

        for b in range(len(lm)):
            K[lm[a], lm[b]] += Ke[a, b]
    return K

# 5. 边界条件处理与求解 (缩减法)
def solve_reduction(K, F, fixed_dof, fixed_value):
    """缩减法处理位移边界条件, 求解节点位移与约束反力"""
    n_total = len(F)
    all_dof = np.arange(n_total)

    # 自由自由度 (未知位移)
    free_dof = np.setdiff1d(all_dof, fixed_dof)

    # 分块矩阵
    K_FF = K[np.ix_(free_dof, free_dof)]
    K_FE = K[np.ix_(free_dof, fixed_dof)]
    K_EF = K[np.ix_(fixed_dof, free_dof)]
    K_EE = K[np.ix_(fixed_dof, fixed_dof)]

    F_F = F[free_dof]
    d_E = fixed_value

    # 求解未知位移
    d_F = np.linalg.solve(K_FF, F_F - K_FE @ d_E)

    # 重构完整位移向量
    d = np.zeros(n_total, dtype=np.float64)
    d[fixed_dof] = d_E
    d[free_dof] = d_F

    # 计算约束反力
    R_E = K_EE @ d_E + K_EF @ d_F - F[fixed_dof]

    return d, R_E, free_dof

# 6. 后处理: 单元应力与轴力
def compute_element_results(model, LM, d):
    """计算所有单元的长度、方向余弦、应力、轴力"""
    nel = model['nel']
    E = model['E']
    A = model['CArea']
    nsd = model['nsd']

```

```

results = []
for e in range(nel):
    Ke, L, c, s = compute_element_stiffness(e, model)
    lm = LM[e]
    de = d[lm] # 提取单元节点位移

    # 应力计算
    if nsd == 1:
        B = np.array([-1, 1]) / L
    else:
        B = np.array([-c, -s, c, s]) / L
    sigma = E[e] * B @ de
    N = sigma * A[e] # 轴力

    results.append({
        'element_id': e + 1, # 输出转回1 起始
        'L': L,
        'c': c,
        's': s,
        'sigma': sigma,
        'N': N
    })
return results

# 7. 结果输出与校验
def print_results(model, K, d, R_E, elem_results, LM):
    print("=" * 60)
    print(f"算例名称: {model['title']}")
    print("=" * 60)

    # 1. 总体刚度矩阵
    print("\n1. 总体刚度矩阵 K (施加边界条件前): ")
    np.set_printoptions(precision=6, suppress=True)
    print(K)

    # 2. 对称性检查
    is_symmetric = np.allclose(K, K.T, atol=1e-12)
    print(f"\n2. 对称性检查: {'通过' if is_symmetric else '不通过'}")

    # 3. 奇异性检查
    det_K = np.linalg.det(K)
    print(f"3. 奇异性检查 (行列式值): {det_K:.6e}")

```

```

print(f"    矩阵{'奇异' if abs(det_K) < 1e-9 else '非奇异'}")

# 4. 对号矩阵 LM
print("\n4. 对号矩阵 LM (0 起始编号): ")
print(LM)

# 5. 节点位移
print("\n5. 节点位移结果: ")
ndof = model['ndof']
nnp = model['nnp']
for i in range(nnp):
    if ndof == 1:
        print(f"    节点{i + 1}: d = {d[i]:.6f}")
    else:
        print(f"    节点{i + 1}: u = {d[i * ndof]:.6f}, v = {d[i *
ndof + 1]:.6f}")

# 6. 约束反力
print("\n6. 约束反力: ")
fixed_dof = model['fixed_dof']
for i, dof in enumerate(fixed_dof):
    print(f"    自由度{dof + 1}: R = {R_E[i]:.6f}")

# 7. 单元结果
print("\n7. 单元计算结果: ")
print(f"{'单元':<4}{'长度':<10}{'c':<10}{'s':<10}{'应力':<12}{'轴力':<10}")
for res in elem_results:
    print(

f"{res['element_id']:<4}{res['L']:<10.6f}{res['c']:<10.6f}{res['s']:<
10.6f}{res['sigma']:<12.6f}{res['N']:<10.6f}")
    print("=" * 60)

# 主程序入口
def main(json_path):
    # 1. 前处理
    model = load_model(json_path)

    # 2. 生成对号矩阵
    LM = build_LM(model['IEN'], model['ndof'], model['nen'])

    # 3. 单元分析: 计算所有单元刚度矩阵

```

```

Ke_list = []
for e in range(model['nel']):
    Ke, _, _, _ = compute_element_stiffness(e, model)
    Ke_list.append(Ke)

# 4. 组装总体刚度矩阵
K = assemble_global_stiffness(LM, Ke_list, model['n_total_dof'])

# 5. 边界条件处理与求解
d, R_E, _ = solve_reduction(K, model['F'], model['fixed_dof'],
model['fixed_value'])

# 6. 后处理
elem_results = compute_element_results(model, LM, d)

# 7. 输出结果
print_results(model, K, d, R_E, elem_results, LM)

if __name__ == "__main__":
    case_file = "case2_2d_truss.json"
    if os.path.exists(case_file):
        main(case_file)
    else:
        print(f"未找到输入文件 {case_file}")

```